

```
def actions(self, state: Any) -> List[Any]:  
    """Return the legal actions available in the given state."""  
    pass
```

What does this lock of code do?

This block declares the `actions` method inside the abstract `Problem` class. Line by line:

```
def actions(self, state: Any) -> List[Any]:
```

Defines a method named `actions`. It takes `self` (the object itself) and `state` (the state you're asking about — `Any` means it can be any type). The `-> List[Any]` says it's expected to return a list of any-type items.

```
"""Return the legal actions available in the given state."""
```

A docstring describing the method's job: given a state, tell me which moves are legal from there.

so this is how it should be ?

```
def actions(self, state: Tuple[int, int]) -> List[str]:
    legal_actions = []
    row, col = state
    for action, (dr, dc) in MOVES.items(): # action = "UP", (dr, dc)
        = (-1, 0)
        neighbour = (row + dr, col + dc) # a
        if self.in_bounds(neighbour) and self.is_free(neighbour): # b
            + c
            legal_actions.append(action) # d
    return legal_actions
```

Yes, that's correct. That's exactly how `actions` should look.

Just make sure when you paste it back into your `GridProblem` class, the indentation lines up with the other methods (the `def actions` should sit at the same level as `def is_goal`, `def in_bounds`, etc.), and that the old `raise NotImplementedError` line for `actions` is gone.

Next up are the two easy ones — `result` (TODO 3) and `action_cost` (TODO 4). Want to try those yourself first? They're both short, and the hints in the comments basically hand you the answer.

explain how the whole loop should work

Here's the whole loop, walked through with a real example.

Say the drone is at `(1, 1)` on this grid (`0` = empty, `1` = wall):

	col0	col1	col2
row0	0	0	0
row1	0	0	1
row2	0	0	0

The code:

```
python
legal_actions = []
row, col = state # row = 1, col = 1
for action, (dr, dc) in MOVES.items():
    neighbour = (row + dr, col + dc)
    if self.in_bounds(neighbour) and self.is_free(neighbour):
        legal_actions.append(action)
return legal_actions
```

Before the loop: empty basket `legal_actions = []`, and `row = 1, col = 1`.

Now the loop runs **once per move** — 4 times total.

Loop 1 — "UP", offset (-1, 0)

- `neighbour = (1 + -1, 1 + 0) = (0, 1)`
- In bounds? Yes (row 0 exists). Free? Grid at (0,1) is `0` → yes.
- Both true → append `"UP"`. Basket: `["UP"]`

Loop 2 — "DOWN", offset (1, 0)

- `neighbour = (1 + 1, 1 + 0) = (2, 1)`
- In bounds? Yes. Free? Grid at (2,1) is `0` → yes.
- Append `"DOWN"`. Basket: `["UP", "DOWN"]`

Loop 3 — "LEFT", offset (0, -1)

- `neighbour = (1 + 0, 1 + -1) = (1, 0)`
- In bounds? Yes. Free? Grid at (1,0) is `0` → yes.
- Append `"LEFT"`. Basket: `["UP", "DOWN", "LEFT"]`

Loop 4 — "RIGHT", offset (0, 1)

- `neighbour = (1 + 0, 1 + 1) = (1, 2)`
- In bounds? Yes. Free? Grid at (1,2) is `1` → that's a wall → no.
- `if` fails → skip. Basket stays `["UP", "DOWN", "LEFT"]`

Loop's done (all 4 moves checked). `return legal_actions` hands back `["UP", "DOWN", "LEFT"]`.

So the pattern: start empty, try each move, calculate where it lands, keep it only if it's on the grid AND not a wall, then return whatever survived. The `append` is what grows the basket; the `if` is the filter that decides what gets in.



i genuinely dont understand

Okay. Let me strip it all the way down. Forget the long line for a second.

This function's job is to look at one spot and answer: "is the goal here, or below me?" It answers with two things:

1. the goal node (or nothing)
2. a word: "success", "cutoff", or "failure"

Now — the function can't check the *whole* area itself. So it does a trick: it asks its children to check *their* areas, by **calling itself** on each child. That's recursion: the function uses itself as a helper.

So this line:

```
python
result, result_status = self._recursive_dls(problem, child, limit, metrics, current_st
```

in plain English is just:

"Hey child, you go search your area and report back."

And the child reports back two things, which you catch:

- `result` = the goal node it found (or nothing)
- `result_status` = the word telling you how it went

That's the entire line. You're sending the search down to a child and catching its report.

Think of it like a boss delegating:

- Boss (current node) asks worker (child): "search your section."
- Worker searches, comes back, says: "Status: success. Here's what I found."
- `result` = what they found. `result_status` ↓ "success".

```
def bfs(grid, start, end):
    queue = deque([start])
    visited = set()
    while queue:
        cell = queue.popleft()
        if cell == end:
            return True
        if cell not in visited:
            visited.add(cell)
            for neighbor in get_neighbors(grid, cell):
                queue.append(neighbor)
    return False

def get_neighbors(grid, cell):
    row, col = cell
    neighbors = []
    for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        r, c = row + dr, col + dc
        if 0 <= r < len(grid) and 0 <= c < len(grid[0]) and grid[r][c] != 0:
            neighbors.append((r, c))
    return neighbors
```

whats the problem

The problem is the **indentation on the bottom lines**. Look closely:

- `custom_problem_1`, `custom_results_1`, the closing `]`, and `show_results(...)` all have **one space** in front of them.
- But `custom_grid_1` and `custom_start_1` start at the very left edge (no space).

Python needs these to line up. Since `custom_problem_1` should be at the same level as `custom_start_1`, that one leading space makes Python throw an `IndentationError`.

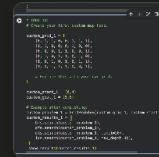
Fix: remove the single space at the start of each of those bottom lines so they're flush left, matching `custom_grid_1` and `custom_start_1`.

Easiest way: click right before the `c` in `custom_problem_1` and press Backspace once to delete that leading space. Do the same for `custom_results_1`, the `]`, and `show_results`. They should all start at the exact left margin.

The lines *inside* the list (`bfs.search(...)`, etc.) stay indented — those are supposed to be indented because they're inside the `[ ]`. It's only the four lines that should be flush-left that currently have the stray space.

So: delete the one leading space from `custom_problem_1`, `custom_results_1`, `]`, and `show_results`. Then run.





Almost fixed — but the last two lines still have leading spaces:

- The closing `]` of `custom_results_1` has a space in front of it.
- `show_results(custom_results_1)` has a space in front of it.

These two need to be flush left, lined up with `custom_results_1 = [`.

Fix:

- Click before the `]` and delete the leading space so it lines up with `custom_results_1`.
- Click before `show_results` and delete the leading space so it's flush left too.

So the bottom should look like:

```
python

custom_problem_1 = GridProblem(custom_grid_1, custom_start_1, custom_goal_1)
custom_results_1 = [
    bfs.search(custom_problem_1),
    dfs.search(custom_problem_1),
    dls.search(custom_problem_1, limit=20),
    ids.search(custom_problem_1, max_depth=40),
]
show_results(custom_results_1)
```

The `]` lines up with `custom_results_1` (left edge), and `show_results` is at the left edge too. The four `.search(...)` lines stay indented inside the brackets — those are correct.

Delete those two stray spaces and run it.

```
total_nodes_expanded
what is this and where is this coming from ?
result.nodes_expanded and this too
```

Two different things — let me separate them.

`total_nodes_expanded` — this is a variable *you create yourself*. It's not given by the lab. You make it to hold the running sum. You invent the name and start it at zero:

```
python
total_nodes_expanded = 0
```

You could call it anything — `total`, `sum_nodes`, whatever. It's just a box you made to keep adding into. The lab's requirement 2 ("accumulate total nodes expanded") is *why* you create it, but the variable itself is yours.

`result.nodes_expanded` — this comes from the DLS run. Here's the chain:

When you call DLS inside your loop:

```
python
result = dls.search(problem, limit=limit)
```

`dls.search(...)` returns a `SearchResult` object (the same kind you've been building all along). You store it in `result`.

That `SearchResult` has fields inside it — remember the class? `algorithm`, `status`, `solution`, `nodes_expanded`, etc. So `result.nodes_expanded` reaches into that result and pulls out its `nodes_expanded` number — how many nodes *that one DLS run* expanded.

Just one line left: `show_results(custom_results_1)` still has a space in front of it. Everything else is now flush left correctly.

`show_results` needs to start at the very left edge, lined up with `custom_results_1` and `custom_problem_1`.

Click right before the `s` in `show_results` and press Backspace once to remove that leading space. Then it'll match:

```
python
]
show_results(custom_results_1)
```

Both the `]` and `show_results` at the left margin.

Fix that one space and run it.

Click right before the `s` in `show_results` and press Backspace once to remove that leading space. Then it'll match:

Fix that one space and run it.


```

def is_goal(self):
    state: Tuple[int, int]
    start
    state: Tuple[int, int] -> bool
    if state is equal to the goal state:
        return True
    else:
        return False

```

There's your `is_goal` bug. You have:

python

```

def is_goal(self, state):
    if state == self.goal:
        return state == self.goal

```

The `return` is inside the `if`. So it only returns when the state matches the goal. When it doesn't match, the function falls off the end and returns `None` — which is why the test `is_goal((0,0)) is False` failed (it got `None`, not `False`).

Fix: delete the `if` line and just return the comparison directly:

python

```

def is_goal(self, state: Tuple[int, int]) -> bool:
    return state == self.goal

```

`state == self.goal` is already `True` or `False` on its own — no `if` needed. When it matches → `True`. When it doesn't → `False`. That's exactly what the test wants.

So your `is_goal` should be just two lines: the `def` line and `return state == self.goal`. Remove the `if state == self.goal:` line entirely.

Fix that, re-run the `GridProblem` cell, then re-run the test cell. It should pass now.



```
n):  
    n, limit=limit)  
    nodes+ttnodes.nodes_  
    size>max_stack_size:  
        max_frontier_size  
    ss":
```

You wrote `ttnodes.nodes_expanded` — there's an extra "s" on `ttnode`. Your variable is `ttnode` (singular), not `ttnodes`.

Look at the line:

```
python  
explored_nodes = explored_nodes + ttnodes.nodes_expanded
```

The `ttnodes` (with s) doesn't exist — everywhere else you use `ttnode` (no s, e.g. `ttnode.max_frontier_size`, `ttnode.status`). The editor even underlined it in red.

Fix to `ttnode` (no s):

```
python  
explored_nodes = explored_nodes + ttnode.nodes_expanded
```

So just delete the extra "s" — `ttnodes` → `ttnode`. Then re-run the `IterativeDeepeningSearch` cell and the results cell. That's the last typo.

📄 ▶ 👍 🗨 ↻





First, that popup is just Colab's privacy notice — click **Continue** to dismiss it, it's unrelated to your error.

Now the actual error: one of the four searches in `custom_results_1` returned `None`. Since BFS/DFS/DLS worked on the sample map, the likely culprit is **IDS** — it has a code path that returns nothing.

Let me have you find which one. Add a temporary cell and run:

```
python

for r in custom_results_1:
    print(r.algorithm if r is not None else "THIS ONE IS None")
```

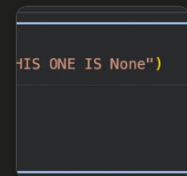
Whichever prints "THIS ONE IS None" is the broken search.

I strongly suspect it's IDS, and here's why: your IDS only returns inside the loop (on success/failure) or after the loop (cutoff). But there's a subtle gap — if `max_depth + 1` makes the loop run, but **none** of your returns fire for some reason, or if there's a stray empty `return`, you get `None`.

The fastest path: paste your **current** `IterativeDeepeningSearch` code (the actual one in the notebook now). I'll check that every branch returns a real `SearchResult` and none returns bare `None`.

But before that — do the simple thing first: click **Continue** on the popup, then **Runtime** → **Run all**. There's a real chance `custom_results_1` is stale (built before you fixed IDS). Run all rebuilds it with the fixed code. If the error clears, you're done. If it persists, paste your IDS code and I'll find the `None`.





```
THIS ONE IS None)
```

It's **DFS** — the second one (BFS, then "THIS ONE IS None", then DLS, IDS). So your `DepthFirstSearch.search` returns `None` on this particular map.

Why DFS specifically? Your DFS returns success/failure inside or after the loop — but there's one path that returns nothing. The most common cause: **DFS found the goal isn't reachable in a way that exits without hitting your failure return**. But more likely, you're missing the failure return entirely, OR the early-exit success has a problem.

Paste your current `DepthFirstSearch` code and I'll find the exact line. But based on what I remember, the likely issue is this: earlier your DFS was missing the **failure return** after the while loop. If the goal isn't reachable on this custom map, the loop empties, falls off the end, and returns `None`.

Check the very end of your `DepthFirstSearch.search` — after the `while` loop, is there this?

```
python
failure = SearchResult(algorithm=algorithm, status="failure", solution=None,
                        nodes_expanded=nodes_expanded, max_frontier_size=max_fr
                        reached_count=len(reached))
return failure
```

If that's missing, that's your `None`. On this custom map, the goal probably *can't* be reached from your start (your map is heavily walled), so DFS runs out of frontier and needs the failure return to report it.

Two things to do:

1. Make sure DFS has the failure return after the while loop. Paste your DFS if unsure.
2. Re-run the `DepthFirstSearch` cell, then run the cell that builds `custom_results_1`, then `show_results`.

im done with the work....is it correct?

Let me read through your notebook and check everything.

Ran 2 commands >

Good — no leftover `raise NotImplementedError` anywhere. Now let me actually run the whole notebook to confirm it all works end to end.

Ran 2 commands >

Read-only directory — let me copy to a writable location and run there.

Run notebook in writable home dir >

It ran all the way through with **no errors** — every cell executed cleanly. Now let me pull out the actual results tables so we can see the algorithms produced sensible output.

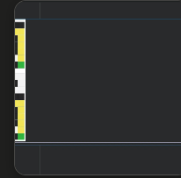
Extract results tables and any errors from executed notebook >

Your notebook **runs cleanly with no errors**, and the algorithms are working correctly. Here's my read on the results.

The sample map (Cell 14) — this is the important one, and it looks great:

Algorithm	Status	Solution depth	Nodes expanded	Max frontier
BFS	success	18	50	5
DFS	success	18	18	4
DLS (limit 10)	cutoff	—	16	11
IDS	success	18	356	19

This is exactly the kind of result the lab wants you to observe, and it shows your algorithms behave correctly:



why are both diagrams the same? is that how it should be ?

Good question — and no, they're *not* identical, though they look very similar here. Both diagrams are plotting the **sample map** (Cell 23 — the Section 13 visualise cell), one for BFS and one for DFS. On this particular map, BFS and DFS happen to find the **same path**, which is why they look nearly identical.

This can happen. On a map where there's essentially one main corridor to the goal, both algorithms end up taking the same route. Remember from your results table earlier — both BFS and DFS reported solution depth 18 on the sample map. Same depth, same path. So the diagrams matching is correct here, not a bug.

When would they differ? On a map with **multiple possible routes**, BFS (explores wide, finds shortest) and DFS (explores deep, follows one branch as far as it goes) would pick different paths — and the yellow lines would look different. The sample map just doesn't give them much choice.

Now — the more important thing: these two diagrams are for the **sample map**, not your custom maps. The cell below (Cell 22, TODO 10) still has the **old unsolvable grid** — I can see `[0, 1, 1, 0, 0, 0, 1, 1]` etc., which is the version that fails. You haven't yet pasted the solvable corridor grid or the `plot_path` lines for your custom map.

how do i go about the adding the plot path

You add a `plot_path(...)` call after your `show_results(...)` line in the custom map cell. It needs four things: the grid, the start, the goal, and the path to draw.

Here's the line to add to your TODO 10 cell (after `show_results(custom_results_1)`):

python

```
bfs_custom_1 = custom_results_1[0]
plot_path(custom_grid_1, custom_start_1, custom_goal_1,
          path=bfs_custom_1.path, title="Custom Map 1 - BFS Path")
```

Breaking it down:

- `bfs_custom_1 = custom_results_1[0]` — grabs the **BFS result** from your results list. `custom_results_1` is the list `[bfs, dfs, dls, ids]`, so `[0]` is BFS.
- `plot_path(...)` — the function that draws the diagram. You pass:
 - `custom_grid_1` — the map
 - `custom_start_1` — start cell (red)
 - `custom_goal_1` — goal cell (green)
 - `path=bfs_custom_1.path` — **this is what draws the yellow path**. `.path` pulls the list of cells from the result.
 - `title="..."` — the heading shown above the diagram

```
Traceback (most recent call last):
  File "/home/.../ipykernel_3935.py", line 10, in <cell line: 8>:
    sample_iter_complexity:
    custom_problem_1 = GridProblem(custom_grid_1,
    _1, custom_goal_1)
    custom_goal_1 = [
    bfs_search(custom_problem_1),
    ]
NameError: name 'GridProblem' is not defined

An error occurred.
or second custom map here.
```

`NameError: name 'GridProblem' is not defined` again — same stale-kernel issue. The kernel restarted (it's now `ipykernel_3935`), so all your class definitions are gone from memory. You ran this one cell in isolation, but it depends on `GridProblem`, `bfs`, `dfs`, etc. being defined first.

The fix: **Runtime** → **Run all**. This runs every cell top to bottom, re-loading `GridProblem` and all the algorithms, so by the time it reaches your TODO 10 cell, everything it needs exists.

Don't run just the TODO 10 cell on its own — it can't work without the cells above it loaded.

So:

1. Click **Runtime** → **Run all** (or the "Run all" button near the top).
2. Let it run through every cell.
3. Watch for any cell that throws a red error — if one does, that's the cell to fix. But the `NameError` itself just means "the definitions above haven't been run yet in this session."

After Run all, your TODO 10 cell should produce the table + two diagrams with no `NameError`.


```
def show_results(custom_results):
    """
    Parameters:
    custom_results: list of lists
    """
    for result in custom_results:
        for path in result:
            plot_path(custom_grid_1, custom_start_1, custom_goal_1, path=path, title=f"Path: {path}")
    plt.show()

# Example usage
custom_results_1 = [
    [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    ],
    [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    ],
]

show_results(custom_results_1)
```

Typo — `custom_result_1` is missing the s. Line 28 says:

```
python

bfs_result_1 = custom_result_1[0]    # ← missing s
```

Your list is called `custom_results_1` (plural, with s), not `custom_result_1`. You used the right name on line 26 (`show_results(custom_results_1)`) but dropped the s on lines 28 and 29.

Fix both:

```
python

bfs_result_1 = custom_results_1[0]    # add the s
dfs_result_1 = custom_results_1[1]    # add the s
```

Also I notice line 30 has `custom_goal` (should be `custom_goal_1`) — check that too:

```
python

plot_path(custom_grid_1, custom_start_1, custom_goal_1, path=bfs_result_1.path, title=
```

So the fixes:

- `custom_result_1` → `custom_results_1` (lines 28, 29) — add the s
- `custom_goal` → `custom_goal_1` (line 30) — add the _1

```
Traceback (most recent call last):
  File "5/2485694652.py", line 0, in <cell line: 0>()
    l=custom_results[i][0]
    l=custom_results[i][1]
  custom_grid_1, custom_start_1, custom_end_1, custom_path, title='BFS solution Custom Map'
  custom_grid_1, custom_start_1, custom_end_1, custom_path, title='DFS solution Path Custom Map'
NameError: name 'custom_goal' is not defined
```

Two fixes:

Line 30: `custom_goal` → `custom_goal_1` (missing the `_1`). That's the current error.

Line 31: there's a different typo — `path.dfs_result_1.path` should be `path=dfs_result_1.path`. You wrote a **dot** where it needs an **equals sign**. It should be `path=` (assigning the path argument), not `path.` (which reads like accessing an attribute).

So the two `plot_path` calls should be:

```
python
plot_path(custom_grid_1, custom_start_1, custom_goal_1,
          path=bfs_result_1.path, title='BFS solution Custom Map')
plot_path(custom_grid_1, custom_start_1, custom_goal_1,
          path=dfs_result_1.path, title='DFS solution Path Custom Map 1')
```

The fixes:

- Line 30: `custom_goal` → `custom_goal_1`
- Line 31: `path.dfs_result_1.path` → `path=dfs_result_1.path` (dot → equals)

Fix both, re-run the cell. These are just small typos — the structure is right.

```

def main():
    # Create a 15x15 grid with a goal at (14, 14)
    grid = Grid(15, 15)
    grid.set_goal(14, 14)
    # Create a path from (0, 0) to (14, 14)
    path = Path()
    path.add((0, 0))
    path.add((1, 0))
    path.add((2, 0))
    path.add((3, 0))
    path.add((4, 0))
    path.add((5, 0))
    path.add((6, 0))
    path.add((7, 0))
    path.add((8, 0))
    path.add((9, 0))
    path.add((10, 0))
    path.add((11, 0))
    path.add((12, 0))
    path.add((13, 0))
    path.add((14, 0))
    path.add((14, 1))
    path.add((14, 2))
    path.add((14, 3))
    path.add((14, 4))
    path.add((14, 5))
    path.add((14, 6))
    path.add((14, 7))
    path.add((14, 8))
    path.add((14, 9))
    path.add((14, 10))
    path.add((14, 11))
    path.add((14, 12))
    path.add((14, 13))
    path.add((14, 14))
    # Run the IDS algorithm
    ids = IDS(grid, path)
    ids.run()
    # Print the result
    print(ids.result())

```

```

# TODO 10:
# Create your first custom map here.
custom_grid_1 = [
    [0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1,],
    [0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1,],
    [0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1,],
    [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,],
    [1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1,],
    [1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,],

```

Show more

Jun 20   

The `KeyboardInterrupt` means you stopped the cell (or it ran so long you interrupted it) — it was stuck on `ids.search(...)`. That's a sign the search is taking forever, which happens when the goal is unreachable on a big grid: IDS keeps re-running DLS at every depth up to 40, expanding huge numbers of nodes, and on a 15×15 with a walled-off goal it grinds.

The root cause is still the same: your grid's goal **(14, 14)** is on a wall (column 14 is all **1** s), and the grid is chopped up by vertical walls. The searches can never succeed, so IDS thrashes.